

Auto Research Pipeline

老师您好：

我根据您对「auto research pipeline」的要求，实现了一套可复现的实验流水线：用 YAML 定义实验 → 自动执行 → 读取/保存 `metrics.json` → 阈值反馈 → 产物落盘，并配套 CI/CD、GitHub Actions self-hosted runner (GPU)，以及 Claude Code / Vast.ai RTX 5080 的一键路径。以下为项目内容、我对其理解、以及使用方式说明。

一、项目目标

把「做实验」工程化与自动化：配置化、可复现、可验证、可在本机与云端 GPU 一致运行。

二、仓库分支说明 (main 与 exp)

仓库提供两个分支，用途不同，可按需要切换：

分支	定位	适合场景
main	只保留 核心 pipeline (<code>src/auto_research/</code> 、安装配置、CI / GPU workflow、Vast / runner / Claude Code 相关脚本等)，不附带示例实验；实验需在 <code>experiments/</code> 下自行添加 YAML 与训练代码。根目录 README 为英文。	验收「框架 + 自己接入实验」、作为可复用的最小依赖库。
exp	在核心能力之上包含 一个端到端可跑示例 (线性回归 + 梯度下降： <code>experiments/linear_regression.yaml</code> 与 <code>scripts/train_linear_regression.py</code>)，README 含 Claude Code 部署与 Vast.ai 5080 一键说明，代码中补充了 英文注释。	克隆后按 README 一键环境 + 立即跑通，便于快速验证流水线是否正常。

切换命令：

```
git checkout main # 仅核心代码，自行添加 experiments/*.yaml
git checkout exp  # 含示例实验，适合演示与试跑
```

三、要求对照 (我实现了什么)

1) 核心代码 (Pipeline)

- 位置：`src/auto_research/`
- 作用：
- 读取实验 YAML (实验配置化)

- 执行 YAML 中的 `command`
- 统一保存运行记录到 `experiments/runs/<run_id>/`
- 读取指标文件 `metrics.json` (或 YAML 指定的 `metrics_path`)
- 生成阈值反馈到 `experiments/feedback/<run_id>_feedback.json`

2) 实验 (YAML)

- 位置 : `experiments/`
- 方式 : 新增 `*.yaml` 即可定义实验
- 模板 : `experiments/README.md`
- 关键字段 :
 - `name` : 实验名
 - `command` : 实际要运行的命令 (训练/评测脚本)
 - `metrics_path` : 指标文件路径 (默认 `metrics.json`)
 - `success_threshold` : 成功阈值 (自动判定 pass/fail)

3) 反馈 (Feedback)

- 输出目录 : `experiments/feedback/`
- 阈值判定规则 :
 - 指标名包含 `loss` / `error` : 越小越好 ($\leq$ 阈值)
 - 其他指标 : 越大越好 ($\geq$ 阈值)
- 反馈内容 : 每个阈值是否通过、实际值、阈值、方向 (`minimize/maximize`) 等结构化信息

4) CI/CD (GitHub Actions)

- CI : `.github/workflows/ci.yml`
- push / PR 自动跑 : `lint + pytest` (保证核心代码质量与可运行性)

5) GitHub Actions Runner (GPU self-hosted)

- Runner 注册脚本 : `scripts/setup_github_runner.sh`
- 在 GPU 机器 (如 Vast.ai 实例) 注册 `self-hosted,gpu runner`
- GPU workflow : `.github/workflows/gpu-research.yml`
- `workflow_dispatch` 手动触发
- 在 GPU runner 上执行 : `auto-research run ...`
- 上传 `experiments/runs` 与 `experiments/feedback` 为 artifact

6) 一键部署/运行 (Claude Code & Vast.ai RTX 5080)

Claude Code / 本地一键 :

- `CLAUDE.md` : 给 Claude Code 的项目说明与常用命令

- `scripts/claude_code_bootstrap.sh` : 创建 venv、安装、执行 (按 YAML 跑实验)

Vast.ai RTX 5080 一键建实例 :

- `scripts/deploy_vast_5080.sh` : 用 vastai CLI 搜索/创建 RTX 5080 (或通过 `VAST_GPU_QUERY` / `VAST_OFFER_ID` 调整)
- 建好实例后 : clone 仓库 → 注册 runner → GitHub Actions 触发 GPU workflow

四、我对该项目的理解 (设计动机)

- 实验配置化 (YAML) : 新增/修改实验不改主逻辑, 只改 YAML 与训练命令, 便于批量实验与复现。
- 训练脚本与流水线「契约」: 训练脚本负责产出结构化 `metrics.json` ; 流水线只负责执行与评估, 不耦合训练细节。
- 阈值反馈自动化: 把「是否达标」从人工判断变为自动判定, 提升验证效率与可重复性。
- CI 保证长期可用: 每次提交都自动检查, 避免工具链随时间不可运行。
- GPU self-hosted runner: 让同一套 YAML + pipeline 可以在真实 GPU 节点上运行, 符合研究实验的实际工作流。

五、使用

A. 本地

1. 安装 :

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
```

2. 添加实验 YAML :

在 `experiments/` 下新增 `your_experiment.yaml` (模板见 `experiments/README.md`)。

3. 运行单个实验 :

```
auto-research run --cwd . -e experiments/your_experiment.yaml
```

4. 查看输出 :

- `experiments/runs/<run_id>/` : `manifest.json` / `metrics.json` / `summary.json`
- `experiments/feedback/<run_id>_feedback.json` : 阈值反馈

B. Vast.ai RTX 5080 + GitHub Actions (GPU)

1. 本地创建 Vast 实例：

```
pip install vastai  
vastai set api-key YOUR_KEY  
bash scripts/deploy_vast_5080.sh
```

2. SSH，注册 self-hosted runner：

在 GitHub：Repo → Settings → Actions → Runners → New self-hosted runner 获取 registration token。

执行：

```
export GITHUB_TOKEN=<runner registration token>  
bash scripts/setup_github_runner.sh OWNER/REPO
```

3. 在 GitHub Actions 触发 GPU Research workflow，并把 `experiment` 输入设置为你添加的 YAML 路径。

六、补充

Vast marketplace 的 GPU 型号命名/库存可能变化，因此脚本支持通过 `VAST_GPU_QUERY` 放宽搜索条件，或用 `VAST_OFFER_ID` 直接指定。